

AI Engineer Journey — Terimler ve Kısa Kullanım Notlarım

Repo çalışmalarından çıkarılmış kişisel başvuru notu · 9 Ağustos 2026

Bu belgede yalnızca repoda çalıştığım veya deneyle kullandığım kavramlar var. Her maddede terimin kısa anlamı, nerede işe yaradığı ve küçük bir örnek bulunuyor. Amaç formül ezberlemek değil; ileride bir ML, LLM veya RAG sistemi kurarken doğru kavramı doğru yerde hatırlamak.

1. Yazılım ve çalışma düzeni

- **CLI / terminal:** Programları grafik arayüz yerine komutlarla çalıştırdığım ortamdır. Otomasyon, test ve sunucu işleri için kullanılır. Örnek: `pytest -q` ile tüm testleri tek komutla çalıştırmak.
- **Mutlak ve görelî yol:** Mutlak yol dosyanın sistemdeki tam adresidir; görelî yol bulunduğum klasöre göre hesaplanır. Örnek: `/home/alperen/ai-engineering-journey` mutlak, `docs/report.md` görelî yoldur.
- **Shell script:** Tekrar eden terminal komutlarını bir dosyada toplar. Kurulum veya kontrol adımlarını unutmayı azaltır. Örnek: haftalık testleri çalıştıran `run_week1_checks.sh`.
- **Virtual environment:** Python paketlerini sistemden ve diğer projelerden ayırır. Sürüm çakışmasını önler. Örnek: bu projenin `.venv` ortamına `sentence-transformers` kurulması.
- **Dependency:** Kodun çalışmak için kullandığı dış pakettir. Sürümü ve amacı bilinmelidir. Örnek: PDF metni okumak için `pypdf`, embedding için `sentence-transformers`.
- **Module / package / import:** Module tek Python dosyası, package ilişkili modüllerin klasörüdür; import bunları kullanıma alır. Örnek: `labs.rag.reranker` modülündeki sınıfı PDF deneyinde kullanmak.
- **OOP:** Veriyi ve o veriyle ilgili davranışları sınıflarda toplama yaklaşımıdır. Karmaşık AI uygulamalarında bileşen sınırlarını netleştirir. Örnek: bir `Milestone` nesnesinin kendi ilerleme doğrulamasını taşıması.
- **Dataclass:** Veri taşıyan Python sınıflarını daha kısa ve açık yazmayı sağlar. Örnek: arama sonucunda `chunk_id`, metin ve skoru birlikte tutan sonuç nesnesi.
- **Enum:** Bir alanın alabileceği sınırlı değerleri tanımlar. Yazım hatalarını ve geçersiz durumları azaltır. Örnek: artifact türünün rastgele metin yerine önceden tanımlı değerlerden seçilmesi.
- **Validation:** Veri sisteme girmeden önce kurallara uygunluğunu kontrol eder. Hatanın daha erken ve anlaşılır çıkmasını sağlar. Örnek: `top_k` değerinin sıfırdan büyük olmasını zorunlu tutmak.
- **Unicode / text normalization:** Metni karşılaştırmadan önce büyük-küçük harf, noktalama ve Unicode birleşik işaretlerini tutarlı hale getirmektir. Türkçe `İlk` ifadesinin casefold sonrası `i` + nokta işaretine ayrılması, evaluator'ın doğru cevabı yanlış saymasına yol açtı; bu yüzden normalizasyon da test edilir.
- **Type hint:** Fonksiyonun hangi veri tipini alıp döndürdüğünü açıklar. Büyük pipeline'larda yanlış bileşen bağlantılarını erken fark ettirir. Örnek: arama fonksiyonunun `list[ChunkSearchResult]` döndürmesi.

- **mypy:** Kodu çalıştırmadan type hint uyumsuzluklarını kontrol eder. Örnek: reranker'ın beklediği aday tipiyle vector store çıktısının uyuştüğunu doğrulamak.
- **Unit test / pytest:** Küçük bir davranışın beklenen sonucu verdiğini otomatik kontrol eder. Örnek: boş aday listesinde reranker'ın boş sonuç döndürmesi.
- **Mock / fake bileşen:** Testte ağır model veya dış servisin yerine kontrollü sahte nesne kullanılır. Test hızlı ve deterministik olur. Örnek: gerçek cross-encoder indirmeden PDF reranking veri akışını test etmek.
- **Refactor:** Davranışı değiştirmeden kodun yapısını iyileştirmektir. Örnek: tekrar eden validation kurallarını ortak yardımcı fonksiyona çıkarmak.
- **Logging ve debugging:** Logging çalışma sırasında olayları kaydeder; debugging hatanın nerede ve neden oluştuğunu araştırır. Örnek: model yüklenirken veya retrieval sonucu beklenmedik olduğunda ara değerleri görmek.
- **Profiling ve benchmark:** Profiling zamanın hangi fonksiyonlarda harcadığını, benchmark ise belirli işlemin süresini ölçer. Örnek: saf Python matris çarpımıyla NumPy sürümünü karşılaştırmak.
- **Git branch / commit / Pull Request:** Branch değişikliği ana koddan ayırır, commit anlamlı bir kayıt oluşturur, PR değişikliğin incelenip birleştirilmesini sağlar. Bu geçmiş yapılan işi kanıtlar ve geri izlemeyi kolaylaştırır.
- **Conventional Commit:** Commit mesajını `type(scope): açıklama` biçiminde standardize eder. Örnek: `feat(rag): rerank PDF retrieval candidates.`
- **Artifact ve reproducibility:** Artifact yapılan işin görünür çıktısıdır; reproducibility aynı ayarlarla deneyin yeniden üretilebilmesidir. Kod, test, JSON sonucu ve raporu birlikte saklamak bu nedenle önemlidir.

2. Matematik, veri ve hazırlama

- **Scalar, vector ve dimension:** Scalar tek sayıdır; vector sıralı sayılar listesidir; dimension listedeki sayı adedidir. Örnek: bir cümle embedding'i 384 sayıdan oluşuyorsa 384 boyutludur.
- **Dot product:** İki vektörün karşılıklı elemanlarını çarpıp toplar. Benzer yönleri ölçen cosine similarity hesabının temel parçalarından biridir.
- **Norm:** Vektörün uzunluğunu hesaplar. Cosine similarity'de vektör büyüklüğünün sonucu yanıltmasını önlemek için kullanılır.
- **Cosine similarity:** İki vektör arasındaki açının benzerliğini ölçer; yön aynılaştıkça skor 1'e yaklaşır. RAG'de soru embedding'ile chunk embedding'ini karşılaştırmak için kullanılır. Yüksek skor tek başına "bu parça soruyu cevaplar" demek değildir.
- **Matrix:** Satır ve sütunlardan oluşan sayı tablosudur. Transformer ağırlıkları ve toplu embedding işlemleri matris işlemleriyle yürür.
- **Transpose ve matrix multiplication:** Transpose satırlarla sütunları değiştirir; matris çarpımı bir temsili başka boyuta dönüştürür. Attention'daki query-key skorları da matris çarpımlarıyla hesaplanır.
- **Sparse vector:** Elemanlarının çoğu sıfır olan vektördür. Kelime tabanlı TF-IDF gösterimi buna örnektir; sözlük büyür ama bir metin yalnız az sayıda kelime içerir.
- **Dense vector:** Çoğu elemanı dolu olan sabit boyutlu vektördür. Sentence Transformer'ın ürettiği 384 sayılık cümle embedding'i buna örnektir.

- **DataFrame ve Series:** pandas'ta DataFrame satır-sütun tablosu, Series tek sütunluk dizidir. CSV yükleme, eksik değer kontrolü ve feature hazırlamada kullanılır.
- **Raw ve processed data:** Raw data kaynaktan geldiği ilk haldir; processed data temizlenip model kullanımına hazırlanmıştır. Ham veri doğrudan modele verilmemelidir.
- **Missing value ve imputation:** Missing value eksik veridir; imputation bunu bir kuralla doldurur. Örnek: Titanic yaş bilgisini genel ortalama yerine unvan, sınıf ve cinsiyet grubuna göre tahmin etmek.
- **EDA:** Exploratory Data Analysis, model kurmadan önce veri dağılımını, eksikleri ve ilişkileri incelemektir. Amaç "veri ne söylüyor?" sorusuna raporla cevap vermektir.
- **SQL:** Veritabanından doğru satırları seçmek, filtrelemek, sıralamak ve gruplamak için kullanılır. Örnek: `GROUP BY` ile bir grubun ortalama başarı değerini bulmak.
- **Feature (X) ve target (y):** Feature modelin girdisi, target öğrenmeye çalıştığı sonuçtur. Titanic'te yaş, sınıf ve cinsiyet X; hayatta kalma y'dir.
- **Train/test split:** Verinin bir bölümü öğrenme, görülmemiş bölümü değerlendirme için ayrılır. Test verisine bakarak karar vermek gerçek performansı bozar.
- **Numerical ve categorical feature:** Numerical sayısal ölçüm, categorical sınırlı kategori bilgisidir. Yaş sayısal; biniş limanı veya unvan kategoriktir.
- **Encoding ve scaling:** Encoding kategorileri modele uygun sayılara dönüştürür; scaling sayısal sütunları karşılaştırılabilir ölçeğe getirir. Logistic Regression pipeline'ında preprocessing parçasıdır.
- **Pipeline:** Veri hazırlama ve model adımlarını aynı sırada bağlar. Örnek: eksik doldurma → one-hot encoding → model. Eğitim ve tahminde aynı işlemlerin uygulanmasını sağlar.
- **Data leakage:** Modelin normalde tahmin anında bilemeyeceği bilgiyi eğitimde görmesidir. Örnek: tüm veri üzerinde imputation hesaplayıp sonra cross-validation yapmak skoru yapay olarak yükseltebilir.

3. Makine öğrenmesi ve deney değerlendirme

- **Baseline:** Daha gelişmiş yöntemleri kıyaslamak için kurulan ilk basit çalışan modeldir. İyileştirmenin gerçekten değerli olup olmadığını baseline'a göre ölçeriz.
- **Classification:** Girdiyi sınıflardan birine atama problemidir. Örnek: öğrencinin başarılı/başarısız veya Titanic yolcusunun hayatta kaldı/kalmadı tahmini.
- **Logistic Regression:** Sınıf olasılığı üreten sade ve yorumlanabilir bir baseline modelidir. Karmaşık modelden önce veri pipeline'ının çalıştığını göstermek için uygundur.
- **Random Forest:** Birçok karar ağacının tahminini birleştirir. Doğrusal olmayan ilişkileri yakalayabilir; fakat daha büyük ve daha az yorumlanabilir olabilir.
- **Histogram Gradient Boosting:** Hataları sırayla düzelten ağaç tabanlı güçlü bir model ailesidir. Titanic model karşılaştırmasında aynı feature setinde en yüksek yerel CV ortalamasını verdi.
- **Accuracy:** Tüm tahminlerin ne kadarının doğru olduğunu ölçer. Sınıflar dengesizse tek başına yanıltıcı olabilir.
- **Precision:** Modelin pozitif dediği örneklerin ne kadarının gerçekten pozitif olduğunu ölçer. Yanlış cevap vermenin pahalı olduğu RAG no-answer kararlarında önemlidir.

- **Recall:** Gerçek pozitif örneklerin ne kadarını yakaladığımızı ölçer. Cevaplanabilir soruları gereksiz reddetmemek istediğimizde önemlidir.
- **F1 score:** Precision ve recall arasında denge kurar. Tek metrik kullanmak gerekirse iki hata türünü birlikte gözetir.
- **True/false positive ve true/false negative:** Kararların dört temel sonucudur. RAG örneği: kaynakta olmayan hava durumu sorusuna cevap vermek false positive; kaynakta olan venv sorusunu reddetmek false negative'dir.
- **Cross-validation:** Veriyi farklı train/validation parçalarına bölerek modeli birkaç kez ölçer. Tek şanslı bölünmeye güvenmeyi azaltır.
- **Stratified K-Fold:** Her fold'da sınıf oranlarını yaklaşık koruyan cross-validation türüdür. Titanic gibi sınıflandırma problemlerinde kullanıldı.
- **Repeated cross-validation:** Cross-validation'ı farklı bölünmelerle tekrarlar. Küçük veri setinde sonucun ne kadar kararlı olduğunu daha iyi gösterir.
- **Mean ve standard deviation:** CV mean ortalama performansı, CV std fold'lar arasındaki oynaklığı gösterir. Bir modelin ortalaması yüksek ama std'si çok büyükse kararsız olabilir.
- **Random seed:** Rastgele işlemleri tekrar üretilebilir yapar. Örnek: `random_state=42` kullanarak aynı veri bölünmesini elde etmek.
- **Feature engineering:** Ham sütunlardan probleme daha anlamlı yeni özellikler üretmektir. Örnek: isimden `Title`, aile üyelerinden `FamilySize` oluşturmak.
- **Hyperparameter:** Modelin veriden öğrenmediği, deneyi yapanın seçtiği ayardır. Örnek: ağaç derinliği, regularization gücü veya retrieval'daki top-k.
- **Ablation study:** Bir özelliği veya bileşeni çıkarıp sonucun nasıl değiştiğini ölçer. Titanic deneyinde iyileşmenin çoğunu `Title` özelliğinin getirdiği bu şekilde görüldü.
- **Controlled comparison:** Tek seferde yalnız bir değişkeni değiştiririz. Örnek: aynı feature, aynı CV, farklı model; böylece farkın modelden geldiğini söyleyebiliriz.
- **Experiment logging:** Modeli, ayarları, metrikleri ve notları her deneyde kaydetmektir. Aynı deneyi tekrarlamayı ve yalnız iyi sonuçları hatırlama hatasını önler.
- **Structured query trace:** Her query'nin kararını, reason code'unu, aday sayılarını ve stage latency'lerini JSON event olarak kaydetmektir. Raw soru/evidence yerine `question_sha256` kullanarak korelasyon yapılır; amaç hatanın ingestion, retrieval, gate veya LLM katmanını ayırmaktır.
- **Privacy-conscious logging:** Debug için gerekli sinyalleri tutarken ham prompt, kullanıcı sorusu ve belge metnini varsayılan loga yazmamaktır. Hash orijinal metni geri getirmez; yalnız aynı girdileri korele eder. Retention, sampling ve erişim politikası ayrıca belirlenmelidir.
- **Out-of-fold prediction:** Her satır için o satırı eğitimde görmeyen fold modelinin ürettiği tahmindir. Daha dürüst hata analizi ve ensemble çalışmaları için kullanılır.
- **Ensemble:** Birden fazla modelin tahminini birleştirir. Modeller farklı hatalar yapıyorsa tek modelden daha kararlı olabilir; benzer modelleri toplamak otomatik olarak fayda sağlamaz.
- **Error analysis:** Yalnız skora değil, hangi satırların neden yanlış tahmin edildiğine bakmaktır. Yeni feature fikri veya veri sorunu buradan çıkar.
- **Overfitting:** Modelin eğitim veya yerel validation ayrıntılarını ezberleyip yeni veride kötüleşmesidir. Çok sayıda deneyi yalnız public leaderboard'a göre seçmek de leaderboard overfitting oluşturabilir.

- **Negative result:** Beklenen iyileşmenin oluşmaması da sonuçtur. Örnek: yeni feature CV'yi artırmıyorsa saklanır ve neden fayda vermediği raporlanır.
- **Risky flip guard:** Yeni modelin eski tahminden değiştirdiği satırları inceleyen güvenlik kontrolüdür. Toplam skor artmış görünse bile riskli değişiklikleri körlemesine göndermemeyi sağlar.

4. LLM ve Transformer temelleri

- **Tokenizer:** Ham metni modelin sözlüğündeki parçalara böler. Model kelimeleri doğrudan değil, bu token dizisini işler.
- **Subword token:** Kelimeden küçük, karakterden büyük tekrar kullanılabilir parçadır. Türkçedeki ekleri ve sözlükte olmayan kelimeleri yönetmeye yardımcı olur.
- **Token ID:** Tokenın sözlükteki sıra numarasıdır; tek başına anlam taşımaz. Anlamı sayısal temsil embedding tablosundan alınır.
- **Token embedding:** Her token ID'sini öğrenilmiş sayılar dizisine dönüştürür. LLM katmanları bu vektörler üzerinde işlem yapar; sentence embedding ile aynı amaçta değildir.
- **Position bilgisi:** Aynı tokenların farklı sırada oluşunu ayırt etmek için dizideki konumu temsile ekler. “Köpek adamı ısırıldı” ile “adam köpeği ısırıldı” bu nedenle aynı değildir.
- **Transformer:** Token temsillerini attention ve MLP katmanlarıyla tekrar tekrar güncelleyen model mimarisidir. Modern LLM'lerin temelidir.
- **Self-attention:** Her tokenın aynı metindeki diğer görünür tokenlardan ne kadar bilgi alacağını hesaplar. “Onu” gibi bir tokenın hangi önceki kişiye bağlandığını bağlama göre kurmaya çalışır.
- **Query, Key, Value:** Attention'ın üç temsilidir. Query “hangi bilgiye ihtiyacım var?”, Key “bende hangi bilgi var?”, Value “eşleşirsek aktaracağım içerik” şeklinde düşünülebilir.
- **Causal mask:** Sıradaki token tahmin edilirken gelecekteki tokenların görünmesini engeller. Böylece model eğitimde cevabın devamını gizlice okuyamaz.
- **Residual connection, normalization ve MLP:** Residual önceki bilgiyi taşır, normalization sayısal akışı dengeler, MLP her token temsilini dönüştürür. Transformer block yalnız attention'dan oluşmaz.
- **Next-token prediction:** LLM, bağlama göre bir sonraki tokenın olasılıklarını üretir ve seçilen tokenı dizinin sonuna ekler. Akıcı metin üretmesi otomatik olarak doğru bilgi verdiği anlamına gelmez.
- **Context window:** Modelin tek istekte görebildiği toplam token sınırıdır. System prompt, sohbet geçmiş, RAG chunkları, soru ve üretilecek cevap aynı bütçeyi paylaşır.
- **Token budget:** Context window içinde hangi parçaya ne kadar alan ayrıldığını anlatır. Çok fazla ilgisiz chunk eklemek cevaba yer bırakmayabilir ve modeli dağıtabilir.
- **System prompt ve user message:** System uygulamanın genel davranış talimatını, user mevcut isteği taşır. Ancak gerçek öncelik modelin chat template'ine ve uygulama katmanına da bağlıdır; system prompt tek başına güvenlik duvarı değildir.
- **Chat template:** System, user ve assistant mesajlarının modele gönderilmeden önce hangi özel tokenlarla birleştirileceğini belirler. Aynı prompt farklı template'te farklı davranabilir.
- **Base model:** Esas olarak sonraki token tahmini için eğitilmiş ham modeldir. Tamamlama, araştırma ve fine-tuning için uygundur; doğrudan sohbet talimatlarına her zaman iyi uymaz.

- **Instruct model:** Kullanıcı talimatlarını izleme ve sohbet formatında cevap verme için ayrıca eğitilmiş modeldir. Yerel asistan veya RAG cevaplayıcı için genellikle base modelden daha uygundur.
- **Parameter count:** Modelde öğrenilmiş ağırlık sayısıdır. Daha fazla parametre kapasiteyi artırabilir ama “her görevde daha iyi” garantisi vermez; RAM ve gecikme maliyeti de yükselir.
- **Quantization:** Model ağırlıklarını daha düşük hassasiyetle saklayarak RAM kullanımını azaltır. 32 GB RAM'li, GPU'suz bilgisayarda 4B sınıfı modelleri çalıştırmayı kolaylaştırır; küçük bir kalite kaybı olabilir.
- **Temperature ve deterministik ayar:** Temperature yükseldikçe örnekleme çeşitlenir; düşük değer daha tutarlı sonuç verir. Prompt sürümlerini karşılaştırırken aynı ayar ve düşük rastgelelik kullanmak gerekir.
- **Cold ve warm latency:** Cold latency model ilk yüklenirken geçen süreyi, warm latency model bellekteyken sonraki istek süresini anlatır. Yerel model karşılaştırmasında ikisi ayrı ölçülmelidir.
- **Local inference:** Modelin bulut API yerine kendi bilgisayarında çalışmasıdır. Veri kontrolü sağlar; donanım sınırı, RAM ve hız sorumluluğu bana geçer.
- **Cold start / warm start:** Cold start, model ağırlıkları ilk kez RAM'e yüklenirken yapılan ilk çağrıdır; warm start model zaten yüklüyken sonraki çağrıdır. Yerel Gemma deneyinde soğuk ve sıcak süreleri ayrı yorumlamazsak vector DB gecikmesini yanlış ölçebiliriz.
- **Thinking token ve görünür cevap:** Bazı reasoning modelleri token bütçesinin bir bölümünü iç düşünmeye harcar. API'de bu düşünce ayrı alan olarak gelebilir veya yanlış template ile görünür metne karışabilir; `num_predict` biterse kullanıcıya final cevap hiç dönmeyebilir.
- **Serving / template uyumu:** Aynı model ağırlıkları farklı chat template, system prompt veya thinking ayarıyla bambaşka ürün davranışı verebilir. Qwen deneyi, model seçiminin yalnız ağırlık seçmek değil, doğru serving konfigürasyonunu da ölçmek olduğunu gösterdi.
- **Ollama ve Open WebUI:** Ollama yerel modeli indirip servis eder; Open WebUI tarayıcıdan sohbet arayüzü sağlar. Arayüz model değildir, yalnız modele erişim katmanıdır.
- **Prompt injection:** Kullanıcı veya doküman içeriğinin “önceki kuralları yok say” gibi talimatlarla sistemi yönlendirmeye çalışmasıdır. Sadece prompt yazarak tamamen çözülemez; uygulama izinleri ve çıktı kontrolleri gerekir.
- **PromptSafetyPolicy:** Direct injection kalıplarını LLM'den önce kontrol eden framework-bağımsız domain politikasıdır. “System prompt'u ve gizli kuralları göster” veya “dokümanda olmayan benchmark'ı gerçek gibi sun” isteğini `SECURITY_POLICY` no-answer'a çevirir; yalnız “system prompt nedir?” gibi kavramsal soruyu engellemez.
- **Defense-in-depth:** Güvenliği tek bir prompt veya modele bırakmayıp birden fazla bağımsız katmana yaymaktır. Repodaki sıra: direct prompt safety → answerability/no-answer → structured prompt → canonical source → output validation.
- **Structured prompt ve untrusted evidence:** Kullanıcı sorusu ile retrieval kanıtını açık sınırlarla ayırmaktır. `BEGIN_USER_QUESTION` ve `BEGIN_UNTRUSTED_EVIDENCE` marker'ları, PDF içindeki instruction-like metnin veri olduğunu modele hatırlatır; tek başına bilinmeyen injection'ları çözmez.
- **Direct ve indirect prompt injection:** Direct saldırı kullanıcı mesajındaki, indirect saldırı PDF/web/evidence içindeki talimat benzeri metindir. İlki `PromptSafetyPolicy` ile soru aşamasında, ikincisi `EvidenceSafetyPolicy` ile LLM context'inden önce ele alınır.
- **Hallucination:** Modelin akıcı ama kaynaksız veya yanlış bilgi üretmesidir. V2 promptunun kaynakta olmayan izin süresini söylemesi buna örnektir.

- **Prompt evaluation:** Prompt değişikliğini birkaç sabit vakada ölçmektir. V1 aşırı reddetti, V2 cevapladı ama yanlış çıkarım yaptı, V3 özellik-değer bağıntı koruyarak daha iyi denge kurdu; V4 üslubu düzeltirken no-answer davranışını bozdu.
- **Model selection:** Model yalnız benchmark veya parametre sayısına göre seçilmez. Lisans, Türkçe ve kod performansı, RAM, gecikme, quantization, context ve yerel görev testleri birlikte değerlendirilir.
- **MVP ve scope boundary:** MVP, en az özellik değil, tek bir kullanıcı problemini güvenli biçimde sinayacak en dar ürün sürümüdür. Klinik Akış için bu; idari WhatsApp randevu akışı ve insan devralmadır; teşhis, ödeme ve tam hasta kaydı kapsam dışıdır.
- **Shadow mode:** Sistemin gerçek iş akışında öneri veya taslak üretip, sonucu insan onayı olmadan dışarı göndermediği çalışma modudur. Sağlık gibi riskli alanda otomasyondan önce kalite ve hata oranını ölçmek için kullanılır.

5. Embedding, retrieval ve RAG

- **RAG:** Retrieval-Augmented Generation, cevap üretmeden önce dış dokümanlardan kanıt bulup modele bağlam olarak verme yaklaşımıdır. Amaç modelin yalnız ezberine güvenmemektir.
- **Document:** Sisteme giren anlamlı kaynak birimidir. PDF, politika metni veya teknik doküman bir document olabilir; kimlik, başlık, kaynak ve metin taşır.
- **Ingestion:** Kaynağı okuyup normalize ederek RAG sisteminin kullanacağı document/chunk yapısına dönüştürme sürecidir. Örnek: PDF'den seçilebilir metni `pypdf` ile çıkarmak.
- **Structured / section-aware ingestion:** PDF'i yalnız düz metin değil, mantıksal başlık ve bölüm yapısıyla sisteme almaktır. Mentor PDF'inde `04 Yerel modeli...` ve `Teslim Paketi` başlıklarını chunk metadata'sına taşıyınca doğru bölümün adaylara girme oranı iyileşti.
- **Document-specific parser:** Belirli bir belge ailesinin bilinen başlık, tablo veya üst bilgi kurallarını kullanan parser'dır. Güçlü olabilir ama genelleştirilmemelidir; örnek: bu mentor PDF'inin yedi bilinen başlığını configuration olarak vermek ve farklı PDF türünde ayrı test etmek.
- **Chunk:** Uzun dokümanın retrieval için ayrılmış küçük parçasıdır. Chunk her zaman "bir dosya" değildir; dosyanın birkaç cümlesi veya belirli token aralığı olabilir.
- **Overlap:** Komşu chunkların bir miktar ortak metin taşımasıdır. Bilginin chunk sınırında ikiye ayrılmasını azaltır. Örnek: 2 cümlelik chunkta 1 cümle overlap.
- **Metadata:** Chunk metnine eklenen sayfa, başlık, bölüm, tarih veya kaynak bilgisidir. Doğru filtreleme, kaynak gösterme ve daha isabetli retrieval için kullanılır.
- **Chunk-size trade-off:** Küçük chunk daha nokta atışı olabilir ama bağlamı eksiltebilir; büyük chunk daha fazla bağlam taşır ama genel metinle skoru şişirebilir. Mentor PDF'inde 2 cümlelik chunk, amaç sorusunda 5 cümlelik chunktan daha isabetliydi.
- **Vectorization:** Metni matematiksel olarak karşılaştırılabilir sayı temsiline çevirme işlemidir. TF-IDF de embedding de vectorization yapar; çalışma mantıkları farklıdır.
- **Term Frequency:** Bir kelimenin dokümanda kaç kez geçtiğini ölçer. "Python" kelimesi iki kez geçiyorsa o terimin frekansı buna göre artar; anlamdaş kelimeleri anlayamaz.
- **IDF:** Her yerde geçen kelimelerin önemini azaltıp nadir kelimelerin önemini artırır. "ve" düşük, "venv" daha yüksek ayırt edici ağırlık alabilir.

- **TF-IDF:** Kelimenin ilgili belgede sık, tüm koleksiyonda ise nadir olmasını ödüllendiren sparse gösterimdir. Tam teknik terim ve ürün kodu aramalarında güçlüdür.
- **Lexical retrieval:** Kelime veya parça eşleşmesine dayalı aramadır. “python -m venv” gibi exact ifadeleri iyi yakalar; farklı kelimelerle aynı anlamı bulmakta zorlanabilir.
- **Sentence embedding:** Cümle veya paragrafın genel anlamını sabit boyutlu dense vektöre çevirir. Kullandığımız çok dilli MiniLM modeli her metin için 384 sayı üretir.
- **384 dimension:** Cümlelerin 384 ayrı insan-tanımlı özelliği demek değildir. Modelin eğitim sırasında faydalı bulunduğu dağıtık sayısal temsil alanıdır; tek bir sayıya “bu Python anlamıdır” diyemeyiz.
- **Bi-encoder:** Soru ve chunkları birbirinden bağımsız embed eder. Chunk vektörleri önceden hesaplanabildiği için çok sayıda metinde hızlı candidate search yapılır.
- **Semantic search:** Aynı kelimeler geçmese de anlamı yakın metinleri embedding ile bulur. Örnek: “sanal ortam oluşturma” sorgusunun “venv kurulumu” parçasını getirmesi.
- **Vector store:** Chunkları, embeddinglerini ve metadata'yı saklayıp benzerlik araması yapan katmandır. Repoda önce in-memory sürümü kuruldu; Chroma/Qdrant gibi kalıcı ürünler aynı temel ihtiyacı daha büyük ölçekte çözer.
- **Vector database:** Vektörleri ve metadata'yı kalıcı, client/server tabanlı bir collection içinde saklayan vector store türüdür. In-memory liste süreç kapanınca kaybolur; Qdrant gibi bir DB ile PDF chunkları yeniden başlatmada da aranabilir kalır.
- **Collection:** Aynı embedding boyutu ve şemaya sahip vektör kayıtlarının mantıksal grubudur. Örnek: mentor PDF'inin 384 boyutlu chunk embeddingleri için tek bir Qdrant collection.
- **Payload / metadata filtering:** Vektörün yanında saklanan, aramayı daraltan alanlardır. Örnek: yalnız `section_id=local_model` olan chunklarda arama yapmak veya cevaba doğru source bilgisini eklemek.
- **Upsert:** Kimliği daha önce yoksa kayıt ekleme, varsa aynı kaydı güncelleme işlemidir. Mentor PDF'i ikinci kez ingest edilince 48 chunk'ın 96'ya çıkmaması upsert sayesinde.
- **Idempotency:** Aynı işlemi birden fazla kez uygulamanın aynı nihai durumu üretmesidir. İndeksleme hata sonrası tekrar çalıştırılabildiği için RAG ingestion'ında kritik bir güvenilirlik özelliğidir.
- **Persistent volume:** Container silinse veya yeniden başlatılsa bile veriyi container dışındaki bağlı diskte saklayan volume'dür. Qdrant'ın named volume'u restart sonrasında 48 point'in korunmasını sağladı.
- **RAG orchestration:** Retrieval, reranking, context builder, no-answer policy ve LLM çağrısını doğru sırayla birleştiren uygulama katmanıdır. Her bileşen tek başına çalışsa bile bu sıra kurulmazsa kanıtın nerede kaybolduğu anlaşılamaz.
- **Threshold calibration:** “Hangi retrieval skoru yeterli kanıt demektir?” sorusunu sabit vakalarda ölçerek yanıtlama sürecidir. Başka bir veri setindeki 0.50 eşliğini yeni PDF'e doğrudan taşımak false negative üretebilir.
- **Self-hosted ve managed service:** Self-hosted servisi kendi Docker/sunucunda işletmektir; managed serviste sağlayıcı operasyonu yönetir. İlkinde veri kontrolü ve operasyon sorumluluğu, ikincisinde kolaylık ve sağlayıcı/maliyet bağımlılığı trade-off'u vardır.
- **Retrieval:** Soruya en ilgili document veya chunk adaylarını bulma aşamasıdır. İyi LLM yanlış chunklarla doğru cevap veremez; RAG kalitesinin ilk kapısıdır.

- **Top-k:** Retrieval'ın döndüreceği en yüksek skorlu aday sayısıdır. `top_k=5` , en iyi beş chunk sonraki aşamaya geçirir. Çok düşük k doğru kanıt kaçırabilir, çok yüksek k gürültü ve maliyet getirir.
- **Candidate generation:** Hızlı retrieval ile küçük bir aday kümesi oluşturma aşamasıdır. Reranker yalnız bu adaylar arasından seçebilir; doğru parça top-k'ya girmediyse onu geri getiremez.
- **Hybrid retrieval:** Lexical ve semantic skorları birleştirir. Böylece hem exact anahtar kelimeyi hem paraphrase anlamını yakalamaya çalışır.
- **Score normalization ve weight:** Farklı retriever skorlarını aynı aralığa getirip ne kadar etkili olacaklarını ayarlarız. Örnek: `0.6 dense + 0.4 lexical` ; en iyi ağırlık varsayımla değil eval setiyle seçilir.
- **Reranker / cross-encoder:** Soru ve her adayı birlikte okuyup daha hassas relevance skoru üretir. Pahalı olduğu için tüm koleksiyona değil top-k adaylara uygulanır.
- **PDF reranking örneği:** “Teslim paketinde ne var?” sorusunda doğru tablo parçası dense sıralamada 5. adaydı; cross-encoder onu 1. sıraya taşıdı. Yerel model sorusunda doğru bölüm top-5'e girmediği için reranker düzeltmedi.
- **Context builder:** Seçilen chunkları kaynak bilgileriyle düzenleyip LLM'e verilecek bağlamı hazırlar. Sıra, ayırıcılar ve token bütçesi burada yönetilir.
- **Context budget:** LLM'e gönderilecek kanıt için koyduğumuz karakter veya token üst sınırıdır. Gereğinden çok context gecikmeyi artırır ve modelin dikkatini dağıtabilir; az context ise cevabı eksik bırakabilir.
- **Small-to-big retrieval / parent-section expansion:** Küçük child chunklarla doğru yeri bulup, cevap üretirken bağlı olduğu daha geniş parent section'ı kullanma desendir. Yerel model ölçüm sorusunda tek chunk no-answer verirken tam `local_model` section'ı doğru metrikleri içeren cevap üretti.
- **Grounded answer:** Cevabın verilen kanıtı dayandırmasıdır. Model yalnız contextte bulunan bilgiyi söylemeli ve mümkünse kaynağı göstermelidir.
- **Output/evidence validation:** LLM'in ürettiği cevabı kendisine verilen evidence ile üretimden sonra karşılaştıran guardrail katmanıdır. İlk uygulamamız cevapta geçen sayıları evidence'taki sayılarla karşılaştırıyor; örnek: evidence `32 GB` içerirken cevap `64 GB` yazarsa `UNSUPPORTED_NUMBER` warning'i üretir.
- **Unsupported claim:** Cevapta bulunup verilen kanıt tarafından desteklenmeyen sayı veya iddiadır. Bu, retrieval'ın doğru çalışmış olmasına rağmen generation aşamasında hallucination olabileceğini gösterir; ilk sürümde otomatik ret değil, insan/policy incelemesi için structured warning oluşturuyoruz.
- **Canonical source:** Cevabın gerçek dayanağını temsil eden, modelin metninden parse edilmeyen retrieval kaynağıdır. API'deki `sources` listesi `RetrievedChunk` ve Qdrant payload'ından üretildiği için modelin uydurduğu source ID güvenilir kaynak olarak kabul edilmez.
- **Warning ile no-answer farkı:** No-answer, model çağrılmadan önce kanıtın yetersiz bulunmasıdır; warning ise model cevap verdikten sonra şüpheli bir çıktı sinyalinin görünür kılınmasıdır. Warning şu an cevabı otomatik silmez; bu politika validation setinde ayrıca ölçülmelidir.
- **SECURITY_POLICY no-answer:** Soru retrieval'a veya LLM'e gitmeden direct injection olarak reddedildiğinde kullanılan güvenlik reason code'udur. ``NO_EVIDENCE`` kanıt yokluğunu, ``LOW_RELEVANCE`` zayıf benzerliği, ``SECURITY_POLICY`` ise güvenilmeyen talimatı anlatır; bu ayırım trace ve hata analizini netleştirir.

- **EvidenceSafetyPolicy:** Retrieval'dan gelen chunkları LLM prompt'una girmeden önce yüksek güvenli indirect injection kalıpları için kontrol eder. `<|system|>`, “önceki talimatları yok say” veya secret exfiltration gibi parçaları çıkarır; benign prompt açıklamasını korur. Bu filtre de tek başına model güvenliği garantisidir.
- **Security attack matrix:** Bir sistemi tehdit türleri, mevcut kontrol, test kanıtı ve açık kalan işlerle birlikte değerlendiren tablodur. Örnek: direct injection `pass`, ACL filtresi henüz bağlıysa `not_ready`, JSON transport var ama sanitizasyon yoksa `partial` yazılır.
- **Defense in depth:** Güvenliği tek bir savunmaya bırakmamak, katmanlı kontroller kurmaktır. Bu projede PromptSafetyPolicy → EvidenceSafetyPolicy → answerability → structured prompt → output validation zinciri buna örnektir.
- **Fail-closed:** Bir güvenlik kontrolü uygulanamıyorsa isteği başarıyla devam ettirmemektir. Tenant/ACL parametresi geldiğinde 501 `FEATURE_NOT_READY` dönmek veri sızıntısı pahasına sessizce filtrelemekten daha güvenli bir davranıştır.
- **ACL / tenant filter:** Kullanıcının hangi belge veya tenant verisine erişebileceğini belirleyen yetkilendirme filtresidir. Filtre retrieval'dan önce uygulanmalı ve dönen source üzerinde tekrar doğrulanmalıdır; mevcut serviste bu zincir henüz tamamlanmadı.
- **Source provenance:** Bir chunk'ın hangi güvenilir kaynaktan, hangi sürümden ve hangi ingestion sürecinden geldiğini izleyen metadata'dır. Content hash ve pipeline fingerprint kimlik sağlar; signed provenance ve allowlist ise henüz sonraki adımdır.
- **RAG poisoning:** İndekse bilerek yanlış, zararlı veya talimat içeren belge/chunk sokarak retrieval ve generation davranışını bozma saldırısıdır. Stage → verify → activate yarım version'ı gizler; kötü niyetli içeriği tamamen tespit etmez.
- **Output encoding / sanitizer:** Model metnini HTML veya Markdown renderer'a vermeden önce güvenli biçime dönüştürme veya tehlikeli etiket/URL'leri temizleme işlemidir. API'nin JSON dönmesi tek başına sanitizasyon değildir; client tarafında escape gerekir.
- **Rate limit ve backpressure:** İstek hızını ve aynı anda çalışan ağır iş sayısını sınırlayarak kaynak tüketiminin sistemi kilitlemesini önler. PDF byte/page limiti vardır; rate, concurrency quota ve queue backpressure henüz yoktur.
- **Evidence filtering:** Skoru çok zayıf chunkların context'e girmesini engeller. İlgisiz kanıt modelin uydurma yapmasını kolaylaştırabilir.
- **Answerability:** Bulunan kanıtın soruyu gerçekten cevaplamaya yetip yetmediği karardır. Semantik yakınlıkla aynı değildir: “izin süresi” parçası “izin talebi ne zaman yapılır?” sorusuna yakın olabilir ama doğru özelliği cevaplamaz.
- **Threshold:** Cevap vermek için gereken minimum skor sınırıdır. Çok düşük threshold false positive, çok yüksek threshold false negative üretir; sabit eval vakalarıyla ayarlanmalıdır.
- **Score margin:** Birinci ve ikinci aday arasındaki farktır. En iyi skor yüksek görünse bile ikinci adayla fark çok küçükse sistem kararsız olabilir.
- **Answerability profile:** Loose, balanced veya conservative gibi farklı threshold politikalarıdır. Riskli kurumsal kullanımda conservative profil daha çok reddedebilir; yardım odaklı sistem daha dengeli profil seçebilir.
- **No-answer detection:** Yeterli kanıt yoksa cevap üretmek yerine “yeterli bağlam yok” deme yoludur. Similarity skorunu teknik sayıdan ürün kararına dönüştürür.

- **Golden query:** Beklenen doğru document/chunk bilgisi önceden etiketlenmiş test sorusudur. Retriever değişikliklerini aynı sorular üzerinde adil biçimde ölçmek için kullanılır.
- **Rank:** Beklenen doğru sonucun listede kaçınıcı sırada olduğunu gösterir. Rank 1 en iyi durumdur; sonuç top-k içinde yoksa rank bulunamaz.
- **Hit@1 ve Hit@K:** Hit@1 doğru cevabın ilk sırada, Hit@K ilk k sonuç içinde bulunma oranıdır. Reranker kullanılacaksa Hit@K candidate kalitesini, son cevap için Hit@1 hassasiyeti gösterir.
- **Reciprocal Rank ve MRR:** Doğru sonucun sırası r ise reciprocal rank $1/r$ 'dir; MRR tüm soruların ortalamasıdır. Doğru sonucu üst sıralara getiren sistemi ödüllendirir.
- **Retrieval benchmark:** Retriever'ı kolay ve zor golden query setinde düzenli ölçmektir. Yalnız birkaç güzel demo sorusu sistemin güvenilir olduğunu kanıtlamaz.
- **Paraphrase robustness:** Aynı niyet farklı kelimelerle sorulduğunda sistemin davranışını korumasıdır. Dense embedding lexical yöntemine göre bu konuda genellikle daha güçlüdür; yine de ölçülmelidir.
- **Repeatability:** Aynı soru ve ayarlarda sonucun tekrarlar boyunca ne kadar kararlı olduğudur. Yerel Gemma V3 promptu 8 vaka $\times$ 3 tekrarda denenerek küçük ölçeğe kontrol edildi.
- **Phrase coverage:** Bir cevabın beklenen ifade/claim gruplarını içerip içermediğini ölçen offline kalite sinyalidir. Gerçek Gemma smoke'unda beklenen "ilk cevap süresi, toplam süre, bellek" ifadelerinin 0/3'ü bulundu. Exact phrase kullandığı için paraphrase'i kaçırabilir; tek başına doğruluk oracle'ı değildir.
- **Expected phrase ve forbidden phrase:** Expected phrase cevapta bulunması beklenen ifade, forbidden phrase ise güvenlik veya kalite nedeniyle bulunmaması gereken ifadedir. Örnek: model karşılaştırması sorusunda ölçüm ifadeleri expected; kaynakta olmayan bilgiyi uydurma talimatı forbidden olarak etiketlenebilir.

6. Kurduğum zihinsel akış

Veri/ML: ham veri → temizlik → EDA → feature/target → preprocessing pipeline → baseline → cross-validation → feature/model deneyi → hata analizi → kayıtlı karar.

LLM: metin → token → token ID → token + position embedding → Transformer katmanları → sıradaki token olasılıkları → üretilen cevap.

RAG: kaynak → ingestion → document → chunk + metadata → sparse/dense embedding → vector store → top-k retrieval → hybrid/reranking → evidence filtering → answerability → context → LLM cevabı veya no-answer.

Genel mühendislik kuralım: Çalışan çıktı tek başına yeterli değildir. Varsayımı yaz, ölçümü sabitle, hatayı saklama, başarısız deneyi kaydet, testi çalıştır ve kararı kanıtlarla.